IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Manuscript prepared for submission. Publication dates have not been assigned.

Digital Object Identifier Not assigned

A Profile-Guided Register-Allocation Prototype With Greedy Packing and Local Potential Search

AUTHOR NAME NOT SUPPLIED[1]
[1]Affiliation, institution, city, and country were not supplied.
Corresponding author: not supplied (e-mail: not supplied).

Funding information was not supplied. Replace this note before submission.

ABSTRACT Register allocation maps virtual registers onto a finite physical file and spills the remainder. Profile-guided optimization uses supplied block frequencies so that spill decisions can follow executed paths. A spill cost remains an estimate: it depends on which ranges spill, where loads and stores would occur, and which counts the caller trusts. Graph coloring, linear scan, and learned allocators in production compilers often split ranges, coalesce copies, or rewrite spill code. A static weight can keep a range that is costly in the text and cheap on a hot path, and a one-range search can miss a joint reassignment. This paper documents PGARA, a batch prototype for a small three-address representation. It builds a control-flow graph and may-liveness, forms an interference graph with one live range per virtual register, and scores spills from caller-supplied counts. Static greedy and profile-weighted greedy pack ranges in one pass. A potential search accepts a unilateral move only when a tradeoff between average and worst-profile cost strictly decreases. On the built-in suite, profile weighting changes the spilled range in one case (estimated costs 8 and 101), and the tradeoff changes the potential search from estimated cost 19 to 20. These figures are not measured runtime, and the search is not globally optimal. Splitting, coalescing, and application-level timing remain future work.

INDEX TERMS compiler optimization, interference graph, potential game, profile-guided optimization, register allocation, spill cost

I. INTRODUCTION

REGISTER allocation is a late compiler pass. Values that stay in registers avoid the latency and code size of memory traffic, but a target provides only a finite set of physical registers. When two values are live at the same program point they cannot share a register, so the allocator either assigns distinct registers or spills a value to memory and reloads it at later uses. The project name PGARA expands to profile-guided adaptive register allocation. In the implementation inspected here, "adaptive" does not mean an online controller. The analyzer is a batch procedure: the caller supplies the program, the register count, and one or more execution profiles.

The motivation is the gap between a spill decision and the cost a user can defend. A static count of definitions and uses treats a cold block and a hot block alike. A profile can reverse that preference, but only if every block has an explicit count and the cost model states what a spill means. Production compilers already combine profiles with many other optimizations [1], [2]. Isolating the allocation choice from instruction selection, scheduling, and spill-code insertion makes the estimate easier to audit and easier to misread as a runtime result. PGARA is built to keep that distinction visible.

Classical allocators color an interference graph [3]–[6], scan live intervals in roughly linear time [7]–[9], or solve a structured assignment such as the puzzle formulation of Pereira and Palsberg [10]. Profile information has been used to place spills by expected distance to the next use [11]. More recent work learns allocation policies inside LLVM [12], publishes interference-graph data for learning [13], or searches with an evolutionary encoding [14]. Other papers retarget allocation to GPU register files [15], timing-speculative pipelines [16], [17], application-specific baseband kernels [18], or dual-usage vector registers [19]. Those results are real for their own metrics. They do not, by themselves, say which spilled set is cheaper under a caller-supplied multi-profile estimate on PGARA's intermediate

representation.

The shortcomings that matter for this prototype are concrete. Coloring and scan allocators usually rewrite the program into spill code and may split or coalesce ranges; PGARA does neither, so it cannot claim their code-quality results. A single spill weight hides disagreement among profiles. A search that moves only one live range cannot swap two ranges in one step, even when the swap would look better. Learned allocators require a training corpus and a full compiler backend [12], [13], which this repository does not include.

The implemented technique is therefore a small, inspectable pipeline. After parsing and validation, the analyzer builds a control-flow graph, computes may-liveness, and adds an interference edge wherever two virtual registers are simultaneously live. Spill sites are the live uses and the live definitions of each range. Static greedy orders ranges by a profile-independent site count. Profile-weighted greedy orders them by the sum of estimated dynamic spill operations across the supplied profiles. The potential search starts from a valid assignment, by default the all-spilled assignment, and accepts a feasible change of one range only when the potential in Section III-G strictly decreases. The same assignment is reported through a FastAPI service and a React dashboard so that a zero estimate is visible as data rather than as an empty plot. Suitability is limited to explanation, regression tests, and synthetic cases. The prototype is not a drop-in replacement for a production register allocator.

The paper follows that outline inside the IEEE Access article format. The abstract is one paragraph, within the journal's 150–250 word limit [20]. The longer line-counted abstract outline supplied with the manuscript brief is expanded in this introduction rather than set as a second abstract. Section II discusses verified related work and the gap relative to PGARA. Section III formulates the problem, describes the architecture and modules, and records the equations and algorithms taken from the source. Section IV states the tools, parameters, and results that were actually computed, and it separates those results from illustrations and from experiments that were not run. Section V summarizes the limits and the realistic next measurements.

II. RELATED WORK

This section discusses fifteen scholarly works whose bibliographic records were retrieved and checked. Each paragraph states the proposal, the method and tools named by the source, the result stated in the retrieved abstract or title, and the gap relative to PGARA. Where the retrieved abstract does not give a number, no number is added. None of these experiments was re-run in this repository. Their benchmarks, instruction sets, and cost units differ from the estimated spill operations defined in Section III-G, so this paper does not rank PGARA against them.

Chaitin et al. formulated register allocation as graph coloring [3]. The verified record is the 1981 Computer Languages paper; the retrieved metadata did not include an abstract with experimental figures. A companion paper treats allocation and spilling via graph coloring [4]. The method, as used by later coloring allocators, builds an interference graph and spills when a legal coloring is not available. No spill-count figure from those papers is repeated here because it was not in the metadata retrieved for this manuscript. The gap is that a coloring formulation does not, by itself, define PGARA's multi-profile estimate or its one-range search.

Briggs, Cooper, and Torczon improved Chaitin-style coloring with optimistic coloring and a broader treatment of rematerialization [5]. The work is a compiler implementation study. The abstract reports that optimistic coloring decreases the number of procedures that require spill code and reduces spill code when spilling remains necessary, and that rematerialization lowers the cost of spilling some values. Exact percentages were not in the retrieved abstract, so they are omitted. PGARA does not rematerialize values and does not report procedure-level spill-code size.

George and Appel interleaved coloring reductions with conservative coalescing [6]. The method targets copy elimination: nodes that do not interfere may be merged, but unrestricted merging can create spills. The abstract states that Briggs's conservative heuristic leaves too many moves and that iterated coalescing addresses that conservatism. No new runtime number is taken from the paper here. PGARA does not coalesce copies, and its cost model ignores moves between physical registers unless the spill set changes.

Poletto and Sarkar replaced coloring with linear scan [7]. The allocator visits live ranges in a linear pass rather than building a full coloring search. The abstract reports that the algorithm is considerably faster than graph coloring and produces code almost as efficient, with dynamic compilation among the intended uses. No percentage from that evaluation is copied beyond those words. PGARA's greedy pass is also a single packing pass, but it still builds an interference graph and does not allocate by interval endpoints.

Traub, Holloway, and Smith compared linear scan with graph coloring where compile time matters [8]. The abstract describes a linear sweep over register candidates and contrasts it with coloring in optimizing compilers. The retrieved abstract is truncated before any numeric table, so no speed or code-quality delta is stated here. The gap is the same metric mismatch: their comparison is compile time and generated-code performance, not a caller-supplied multi-profile spill estimate.

Wimmer and Mössenböck added lifetime holes and interval splitting to linear scan in the Java HotSpot client compiler [9]. The method splits intervals when register pressure is high and is aimed at just-in-time compilation. The retrieved abstract does not include a numeric speedup, so none is reported. PGARA keeps one live range per virtual register and has no splitting pass; the range identifier is reserved for a later split, but that pass is not implemented.

Pereira and Palsberg modeled the register file as a puzzle board and variables as pieces, including precoloring and register aliasing [10]. For PowerPC, x86, and StrongARM

the puzzles are solved in polynomial time, with a heuristic for spilling. On SPEC CPU2000, the abstract reports compilation time as fast as the extended linear scan used by LLVM in that setting, and x86 code of similar quality. PGARA does not model aliasing or paired registers, and it was not run on SPEC.

Burroughs used expected distance to the next use, computed from branch probabilities, and then placed spill instructions with block frequencies [11]. The implementation is a register allocator evaluated against LLVM on SPEC CINT2000 for ARMv7. The abstract reports average reductions, versus LLVM, of more than 36% and 50% in dynamically executed stores and loads with statically derived profiles, and 50% and 60% with dynamic profiles. Those percentages belong to that LLVM comparison. They are not measurements of PGARA. The gap is that PGARA never emits spill instructions and never counts retired loads; it sums supplied block counts at abstract use and definition sites.

da Silva, Senefonte, and Attrot published an interference-graph dataset for machine-learning register allocation [13]. The abstract identifies graph coloring as NP-complete, notes the shortage of specialized training data, and presents a dataset in response. No model-accuracy figure was in the retrieved abstract fragment, so none is given. PGARA does not train a model and does not export their graph format. The shared idea is only that interference structure is the allocation constraint.

Terci, Abdulhalik, Bayrakci, and Boz proposed BitEA, a bit-encoded evolutionary search for register allocation [14]. The abstract presents bitwise individuals as a way to search assignments when an exact method is too expensive on current processors. The retrieved text does not state a spill-cost delta against a named baseline, so no such delta is claimed. PGARA's search is deterministic best improvement on one range, not an evolutionary population, and it stops at a local test rather than a generation limit chosen for CPU time.

Oh, Jeong, Ro, and Yoon described CASH-RF, a compiler-assisted hierarchical GPU register file that places a small SRAM register cache in front of an STT-MRAM file [15]. The method is a hardware and compiler co-design aimed at energy, using the cache as a write buffer because STT-MRAM writes are costly. The abstract motivates the hierarchy with density and leakage, not with a multi-profile spill formula. PGARA names physical registers `p0`, `p1`, and so on, and it has no cache hierarchy and no energy model.

Xiao, Chen, He, and Yang reallocated registers to loosen read-after-write dependencies on timing-speculative processors [17]. The method builds a read-write dependency aware interference graph and is compared conceptually with no-operation padding. The retrieved abstract states a reduction of timing errors as the goal and does not give a spill-operation count. A related paper uses a dependency-aware interference graph for voltage scaling [16]. Both objectives differ from PGARA's estimated spill operations. A timing-error rate cannot be read as a PGARA cost.

Chen, Liu, and Liu studied register allocation for application-specific processors in 5G micro-baseband kernels [18]. The method exploits known, fixed data sizes and parallel data access rather than a general profile set. The retrieved abstract does not report PGARA's metrics. The gap is the program model: PGARA accepts a textual three-address language with explicit block counts, not a baseband kernel schedule.

Hu, Zhang, Wang, Liang, and Li allocated code that uses array-unit dual-usage register names on SIMD processors [19]. The method combines ordinary register allocation with names that can be read as a whole or lane by lane, so scalar and vector views stay consistent. The abstract describes that consistency problem rather than a profile-weighted spill sum. PGARA's interference relation is pairwise presence at an instruction boundary, with no lane aliasing.

VenkataKeerthy et al. embedded multi-agent reinforcement learning in LLVM for register allocation [12]. The preprint reports an architecture-independent interface and experiments on x86 and AArch64, and its abstract says the results match or outperform the heavily tuned LLVM allocators used in that study. This manuscript does not repeat a numeric speedup from the preprint, and the PGARA tests did not reproduce that LLVM comparison. The gap is both the compiler and the objective: RL4ReAl emits ISA-constrained code, whereas PGARA returns an assignment map and an estimate.

Liu et al. surveyed profile-guided optimization from instrumentation and sampling through compile-time and link-time passes in GCC and LLVM [1]. The preprint is a survey, not a new allocator. It identifies open issues such as sampling overhead, changing workloads, and cross-architecture portability. PGARA does not collect profiles. It refuses to invent a missing block count. That is a narrower contract than a sampling profiler, and it leaves workload drift unmeasured.

Two further sources are used outside this comparison. Monderer and Shapley introduced potential games [21]. PGARA names its objective a potential and accepts only strict decreases, but the source does not contain a proof that every unilateral deviation's payoff change equals the change in that function in the sense of [21]. Lattner and Adve presented LLVM as a lifelong analysis framework [2]. Several allocators above are implemented in LLVM. PGARA is not.

The manuscript does not reach thirty scholarly references. The brief asked for about thirty sources dated 2018–2024 and illustrated an author-year layout. This draft uses IEEE numbered citations, as required by the journal [20], and it keeps earlier register-allocation papers because they are the methods under discussion. It does not add unrelated recent papers to fill the count. Additional register-allocation papers were found, including a 2024 GPU hierarchical-register compiler paper, but no usable abstract was retrieved for that record, so it is not discussed. Oil-spill and marine-allocation papers returned by a broad search are unrelated and are omitted. The shortfall is recorded in the checklist that accompanies this draft.

III. PROPOSED WORK

A. PROBLEM FORMULATION

The input is a program in a restricted three-address language, an integer register count k, and zero or more execution profiles. A program is a sequence of labeled basic blocks. Instructions assign a constant, copy a register, or apply addition, subtraction, multiplication, or division. Terminators are conditional branches of the form `if` register `goto` label, unconditional jumps, and returns. A comparison such as `if t3 > 10 goto B1` is rejected rather than approximated. The false edge of a conditional branch is the next block in source order. The last block must end with a return or a jump. Profiles are not written in the text. Each profile maps every block label to a nonnegative integer execution count. Omitting a label is an error. The analyzer does not substitute zero.

Let R be the set of virtual registers that are live at any instruction boundary. Milestone 1 creates one live range per such register. The allocation key is the range identifier, which currently equals the virtual-register name because ranges are not split. An assignment A maps each range to a physical register p_i with $0 \leq i < k$, or to spill. JSON `null` means spill. An assignment is illegal when two interfering ranges receive the same physical register. The integer k may be zero, in which case every range spills. If k is positive but an interference clique does not fit, the ranges that receive no register spill. That outcome is a result, not an input error. Negative k is rejected. The implemented upper bound is $k \leq 256$.

The objective implemented in source is not a proof of minimum memory traffic. For each spilled range the analyzer estimates how many load and store events the supplied counts imply. Registers that are kept contribute nothing to that sum. Moving a kept range from p_0 to p_1 does not change the estimate. Consequently two different legal packings of the same spill set have the same estimated cost. The potential search optimizes the estimate, not a measured cycle count.

B. SYSTEM ARCHITECTURE

Figure 1 separates the library, the HTTP service, and the dashboard. The library parses text or structured JSON, validates profiles, and runs the analyses in a fixed order. The service exposes health, a sample program, allocation, and a synthetic benchmark. The dashboard can show a fixed illustrative fixture or call the service. Sample mode does not compute an allocation. Backend mode does not substitute the fixture when a request fails.

The default local service address used by the dashboard is http://127.0.0.1:8741. Cross-origin requests are accepted from `localhost` and `127.0.0.1` on any port and are rejected for other origins. Credentials are not enabled. The dashboard's register slider runs from 1 to 16 even though the service accepts 0 through 256. Load cost, store cost, iteration budget, and the game's initial assignment are not separate dashboard controls. Allocation requests send the robustness slider as the tradeoff, start the potential search from the all-spilled assignment, and use iteration budget 200. After the benchmark-request correction in the inspected source, the synthetic suite posts an empty JSON object, so each built-in case keeps the tradeoff and budget stored with that case.

C. MODULES

The parser accepts one instruction per line, # comments, and block headers of the form `block name:`. Structured input uses opcodes `const`, `mov`, `add`, `sub`, `mul`, `div`, `br`, `jmp`, and `ret`. The caller must provide exactly one of source text or structured blocks. Limits enforced by the service are 32 profiles, 500 blocks, 5000 instructions, and 100000 source characters.

Control-flow construction records fall-through, the true branch, the false branch, and unconditional jumps. May-liveness then iterates a backward transfer until block live-in and live-out stop changing. A use that has no definition anywhere in the function is reported as an undefined use. That check is not path-sensitive: a definition on one branch still counts as a definition. A definition whose new value is never live is reported as a dead definition and does not become a store in the spill model. The same virtual register used twice in one instruction contributes one use. The response does not prove that a register is initialized on every path.

Interference is pairwise. If two ranges are both live at an instruction boundary, the graph contains one undirected edge. Nodes and edges are part of the allocation response, together with per-instruction live-in and live-out sets. The dashboard draws the returned edges on a circle layout. That picture is a view of the graph, not a second interference analysis. Degree and the per-instruction liveness lists are returned by the service and are not all rendered.

The cost module is the only producer of the numbers later called estimated spill operations. Static greedy never reads profiles when it orders ranges. If profiles are present, the service still evaluates that static assignment under those profiles so the three algorithms can be tabulated together. Profile-weighted greedy and the potential search require at least one profile. Requesting either algorithm alone without a profile is an input error. Requesting all three algorithms without a profile runs static greedy and marks the other two as skipped.

The dashboard maps the service names `static_greedy`, `profile_greedy`, and `potential_game` to the labels Static greedy, Profile-weighted, and Potential game. The assignment view has its own algorithm menu. The comparison chart does not read that menu. It plots each returned result's own sum. A chart of zeros is therefore a statement that every returned sum was zero. The zero-cost chart change annotates that fact. It does not alter the allocator.

D. FLOW

Figure 2 is the allocation flow implemented by the analyzer. Validation fails closed: unknown labels, negative counts, comparison branches, and extra JSON fields are rejected. Liveness is a fixpoint, not a single backward pass, because

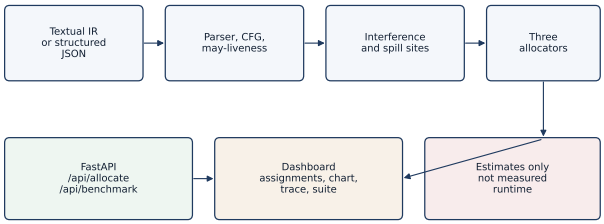


FIGURE 1. System architecture of the PGARA prototype. Estimates returned by the allocators are not measured program runtime.

TABLE 1. Built-in programs and the parameters stored with each case. Counts are the values in the source, not tuned optima.

Case	k	λ	Budget
sequential reuse	1	1/2	10
diamond	1	1/2	20
loop	1	1/2	20
profile vs. static	1	0	10
game average	1	0	10
game worst	1	1	10
zero registers	0	1/2	5
insufficient	1	1/2	20

loops require iteration. The loop sample in the benchmark suite exists for that reason. Interference and spill sites are derived from the fixpoint. Only then do the allocators run. The potential search may call a greedy allocator when its requested start is the static or profile-weighted packing. The dashboard default does not use those starts.

Figure 3 shows the control-flow shape of the built-in tradeoff program. Each edge in that program is an unconditional jump to the next block in the figure. The interesting fact is not a rich branch structure. It is that the estimated cost of spilling `a` is 10 on both supplied profiles, while spilling `b` costs 19 on the profile named skew and 0 on the profile named quiet_b, with unit load and store weights. The new value of `b = b + 1` is unused, so that instruction contributes a load of `b` and not a store. Figure 4 is the corresponding interference graph: `a` and `b` interfere, so one register cannot keep both.

E. BUILT-IN PROGRAMS

The service ships eight synthetic programs. Table 1 records the register count, tradeoff, and iteration budget stored with each case. An empty benchmark request uses those values. The dashboard robustness slider is not applied to them.

Sequential reuse is three assignments in one block. The ranges do not interfere, so one physical register is reused. The diamond is a conditional branch whose false edge is the next block in the text, named `right`, and whose true edge is `left`. The loop is the reason liveness iterates. Its header says `if i goto body`, and the next block in source order is `exit`, so the false edge leaves the loop. The body is reached only by the true edge and jumps back to the header. A single backward pass would not see that `i` and `s` are live around the back edge.

The profile-versus-static program puts three updates of `a` in a block counted once and one update of `b` in a block counted 100 times, then returns `a`. With one register, the static site count and the profile weight order the two ranges differently. The two game cases are the tradeoff program of Fig. 3, one at $\lambda = 0$ and one at $\lambda = 1$, each with budget 10. The profile named skew counts the blocks `def_a`, `def_b`, `use_a`, `use_b`, and `exit` as 2, 9, 3, 10, and 2. The profile named quiet_b uses 2, 0, 3, 0, and 2. Zero registers runs the sequential program at $k = 0$. The insufficient-register case defines four values that are live together at a later addition, with only one physical register.

Figure 5 summarizes the potential search. The scan order is live-range identifier, then $\mathrm{p}_0, \mathrm{p}_1, \ldots$, then spill. Among feasible moves that strictly lower the potential, the search keeps the lowest potential. An equal potential does not replace an earlier candidate and is not accepted. If the iteration budget is zero, no move is accepted. Stopping reasons are a single greedy pass, no improving move, or exhaustion of the budget. The no-improving-move reason is a local equilibrium of this neighborhood. The implementation states that it is not a proof of global optimality.

F. LIVENESS AND SPILL SITES

Block liveness is ordinary backward may-analysis. The live-out set of a block is the union of the live-in sets of its successors. Inside the block, the transfer walks instructions from the end toward the start: a destination is removed from the live set, and every register named as an operand is added. The iteration stops when no block live-in or live-out changes. The implementation also stops and reports an error if the iteration count exceeds $2BN + 2$, where B is the number of blocks and N is the number of mentioned virtual registers, or 1 when the program mentions none. That cutoff is a guard. It is not a complexity theorem, and the tested programs converge before it.

After the block solution exists, the same transfer is stored at each instruction boundary. Interference pairs are collected from those boundaries, including the live-out set at the end of the block. A destination that is live immediately after its

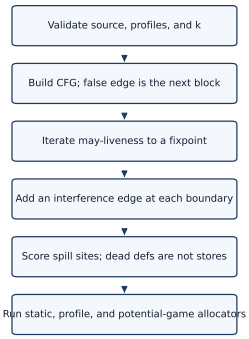


FIGURE 2. Allocation flow. Profile counts are never invented for a missing block.



FIGURE 3. Control-flow graph of the built-in tradeoff program. Edges are jumps in source order.

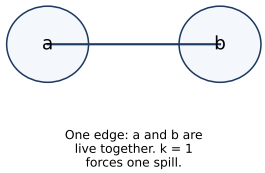


FIGURE 4. Interference graph of the tradeoff program. With one physical register, exactly one of `a` or `b` can be kept.

instruction is recorded as a store site of that virtual register. A destination that is not live is recorded as a dead definition and is not a store site. Each operand occurrence is a load site. The lists are gathered during a backward walk and then reversed so that they appear in source order. If the walk's live set at the start of a block disagrees with the block live-in, the analyzer raises an error instead of returning an assignment. Undefined uses, the names that occur as operands but are never defined anywhere in the function, are listed in the response. That list is not path-sensitive.

The allocation response therefore contains more than the three assignment maps. It includes the control-flow edges, the per-instruction live sets, the interference edges and degrees, the load and store sites, the undefined-use names, and the dead definitions. For the potential search it also includes the accepted moves and the stopping reason. The dashboard draws the interference edges and the assignment each algorithm returned, and it plots each algorithm's own sum of profile costs. It does not recompute liveness in the browser. Fields that the dashboard schema does not display, including the full per-instruction live sets, remain available from the service response.

G. EQUATIONS

The symbols below match the functions in the cost module. All costs are dimensionless estimated spill operations, not seconds and not joules. w_L and w_S are the integer load and store weights. The service defaults are $w_L = w_S = 1$, and each weight is an integer from 0 through 1000. $U(r)$ is the set of use sites of range r, and $D(r)$ is the set of live-definition sites. A dead definition is absent from $D(r)$. For a site s, $\mathrm{block}(s)$ is its basic block. Profile m supplies a nonnegative integer $f_m(b)$ for every block b. M is the number of profiles and is at least one whenever a profile-dependent quantity is evaluated.

The static spill operation count of one range ignores profiles:

$$C^{\mathrm{static}}(r) = w_L|U(r)| + w_S|D(r)|. \tag{1}$$

The estimated dynamic count of spilling r under profile m

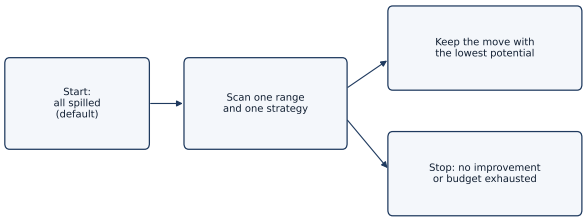


FIGURE 5. Potential search. Only a feasible one-range change that strictly lowers the potential is accepted.

multiplies each site by the count of its block:

$$C_m(r) = w_L \sum_{u \in U(r)} f_m(\text{block}(u)) + w_S \sum_{d \in D(r)} f_m(\text{block}(d)). \tag{2}$$

An assignment A pays only for spilled ranges. The profile cost of the whole assignment is

$$C_m(A) = \sum_{r : A(r) = \text{spill}} C_m(r). \tag{3}$$

Profile-weighted greedy does not order ranges by $C_m(A)$. It orders them by the sum of the per-range estimates, which is independent of the assignment:

$$W(r) = \sum_{m=1}^{M} C_m(r). \tag{4}$$

Static greedy uses $C^{\text{static}}(r)$ in place of $W(r)$. Ties break by range identifier, ascending.

The potential uses the tradeoff $\lambda \in [0, 1]$. The service converts the request value to a rational with denominator at most 10^6 before the search. With $\mathbf{c}(A) = (C_1(A), \ldots, C_M(A))$,

$$\Phi_\lambda(A) = (1 - \lambda) \frac{1}{M} \sum_{m=1}^{M} C_m(A) + \lambda \max_m C_m(A). \tag{5}$$

Thus $\lambda = 0$ uses the average estimated cost and $\lambda = 1$ uses only the worst profile. The average and the maximum are computed with exact rational arithmetic. A candidate assignment A' is accepted only when

$$\Phi_\lambda(A') < \Phi_\lambda(A) \tag{6}$$

holds on those rationals. A floating-point rendering of the potential is for display and is not the comparison.

Feasibility does not depend on Φ_λ. Spill is always feasible. A physical register p is feasible for range r when no interfering neighbor already holds p:

$$\text{feasible}(r, p, A) \equiv \forall n \text{ adjacent to } r,\ A(n) \neq p. \tag{7}$$

Interference itself is existential over instruction boundaries. If $L_{\text{in}}(s)$ and $L_{\text{out}}(s)$ denote the may-live sets at the boundaries of instruction site s, an undirected edge exists when

$$\exists s \text{ such that } \{r, q\} \subseteq L_{\text{in}}(s) \cup L_{\text{out}}(s). \tag{8}$$

Equation (8) is the implemented intent: a pair that is simultaneously live at a boundary interferes. It is may-liveness, so an edge can exist for a pair that no single concrete execution holds at once.

If $w_L = w_S = 0$, every estimate is zero. The potential then never strictly falls, so the search accepts no move from an all-spilled start. Greedy packing can still assign registers, because it does not test Φ_λ. That case is covered by a unit test and is not used in the tables of Section IV.

H. ALGORITHMS

Algorithm 1 is the shared packing procedure. Static greedy sets the weight to (1). Profile-weighted greedy sets it to (4). Both then choose the lowest-numbered free physical register among neighbors that have already been assigned. Ranges later in the order are not yet occupying a register. The pass does not revisit an earlier spill if a later decision would have freed a register. The reported potential, when profiles exist, is computed after packing so that the three results can be compared. Static greedy does not optimize that potential, and profile-weighted greedy does not claim to minimize it either.

Algorithm 2 is best-improvement search over the neighborhood defined by (7). The default initial assignment spills every range, because a cost that depends only on the spill set has no improving one-register move once a maximal greedy packing has filled every feasible register. Starting from greedy can therefore stop immediately even when a two-range exchange would reduce Φ_λ. A unit test locks that behavior for the tradeoff program at $\lambda = 1$ when the search starts from profile-weighted greedy: the search keeps `a` and spills `b`, which is not the spill set chosen from the all-spilled start. The iteration count is the number of accepted moves. Budget exhaustion is not described as optimality.

The two algorithms, the eight equations, and Figures 1–5 are the methodology. They are descriptive of the source. They are not an approximation theorem, a convergence rate,

Algorithm 1 One-pass greedy packing

Require: ranges R, interference edges, register count k, weight $w(r)$
Ensure: assignment A
  $P \leftarrow (\mathrm{p}_0, \ldots, \mathrm{p}_{k-1})$
  order R by decreasing $w(r)$, then by range identifier
  for each r in that order **do**
    $O \leftarrow$ physical registers held by neighbors of r that are already assigned
    if some register in P is absent from O **then**
      $A(r) \leftarrow$ the lowest such register
    else
      $A(r) \leftarrow$ spill
    end if
  end for
  Evaluate (3) and, if profiles exist, (5) on A

Algorithm 2 Potential search from a valid start

Require: ranges, edges, k, profiles, λ, budget B, start mode
Ensure: assignment and stopping reason
  if start mode is all spilled **then**
    $A(r) \leftarrow$ spill for every range r
  else if start mode is static greedy **then**
    $A \leftarrow$ Algorithm 1 with (1)
  else
    $A \leftarrow$ Algorithm 1 with (4)
  end if
  reason $\leftarrow$ budget
  for B iterations, or none if $B = 0$ **do**
    $\phi \leftarrow \Phi_\lambda(A)$ using (5)
    best $\leftarrow$ none
    for each range r in identifier order **do**
      for each strategy in $\mathrm{p}_0, \ldots, \mathrm{p}_{k-1}$, then spill **do**
        if the strategy differs from $A(r)$ and satisfies (7) **then**
          Let A' be A with only r changed
          if $\Phi_\lambda(A') < \phi$ and $\Phi_\lambda(A')$ is the best so far **then**
            best $\leftarrow A'$
          end if
        end if
      end for
    end for
    if *best* is none **then**
      reason $\leftarrow$ no improving move
      break
    end if
    $A \leftarrow$ *best* and record the move
  end for

or a claim that the potential search dominates greedy packing on programs outside the tested set.

IV. IMPLEMENTATION AND RESULT ANALYSIS

A. TOOLS AND SETUP

The analyzer is a Python package, version 0.1.0, checked with the interpreter that produced the result file accompanying this draft, Python 3.12.3 [22]. HTTP models use Pydantic and FastAPI [23], [24]. Tests use pytest. The dashboard is a TypeScript React application built with Vite [25], [26]. No application binary was timed on a named CPU for this paper, and no CPU model is reported, because a controlled timing study was not run. The service records `execution_time_ms` for each allocator call. The unit test for the benchmark endpoint checks that this field is nonnegative. That check is not a performance result.

The commands used to exercise the library are those in the project instructions. From the `backend` directory, create a virtual environment, install `requirements.txt`, and run `python -m pytest`. The service is started with

```
python -m uvicorn pgara.api:app
-host 127.0.0.1 -port 8741
```

The numbers in Section IV-D were obtained by calling the same `analyze` function the service calls. Load cost and store cost were 1. The potential search started from the all-spilled assignment. They are stored in `verified-results.json` next to this manuscript. Rerunning that call on version 0.1.0 reproduces the estimates. It will not reproduce wall-clock time, which was deliberately not saved as a result.

The dashboard sample program has blocks `B0`, `B2`, `B1`, and `B3` in that source order, so the false edge of the branch in `B0` enters `B2`. Its three profiles, in source order, are Hot loop $(1000, 150, 850, 1000)$, Balanced $(1000, 500, 500, 1000)$, and Cold-heavy $(1000, 800, 200, 1000)$. The allocation tradeoff used for that program in the tables is $3/5$, the rational image of the dashboard's default robustness 0.6. The built-in suite does not use that tradeoff. Each case uses the fraction stored in the suite.

No genuine project screenshot is present in the repository. None is inserted. A screenshot of the zero-cost annotation would illustrate the interface only; it would not add an allocation measurement.

B. PARAMETER SETUP

Table 2 lists the parameters that affect the reported estimates and the values used for the two result groups. The table is a record of the run, not a tuned optimum. Load and store weights stay at the defaults because the dashboard does not expose them. Changing either weight scales (2) and can change greedy order. That sensitivity was not swept.

C. METRICS

Five quantities are supported by the implementation and by the runs above. They are defined in Table 3. Estimated spill operations are plotted in Fig. 6 and listed in Tables 4, 5, and 6. Static spill operations and spilled-range identifiers are columns of those tables. Validity and spill-set agreement are recorded in the same tables: every tabulated assignment

TABLE 2. Parameters of the reported runs. Estimated spill operations use dimensionless counts.

Parameter	Dashboard sample	Built-in suite
Register count k	2, 3, 4, 5, 6	Per case, including 0 and 1
Tradeoff λ	$3/5$	The case value
Load and store weights	1 and 1	1 and 1
Game start	All spilled	All spilled
Iteration budget	200	The case value
Profiles	Three supplied maps	The case maps

TABLE 3. Metrics that the reported runs support. Units are counts unless noted.

Metric	What was computed
Estimated spill operations	Sum of (3) over the supplied profiles. Not seconds.
Static spill operations	Sum of (1) over spilled ranges.
Spilled live ranges	Identifiers with $A(r) = \text{spill}$.
Validity	Interference check after assignment. All tabulated runs passed.
Spill-set agreement	Whether the three spill sets were equal on that input.

passed the interference check, and agreement is whether a row names one spill set or three. Separate plots of those two checks would repeat a pass-or-fail observation. They are not drawn. Estimated spill operations are (3) summed across profiles, which is what the dashboard calls dynamic spill operations when profile rows exist. Static spill operations are (1) summed over spilled ranges. Spilled-range count is the size of the spill set. Validity is the post-check that no interfering pair shares a physical register; every result used here passed that check. Agreement records whether the three algorithms returned the same spill set. Agreement is an observation about these inputs. It is not a theorem that the algorithms always agree.

Allocator time is available and is intentionally not a sixth reported metric. One call's milliseconds depend on the host, the interpreter, and other load. No repetition, warmup, or interval was collected. A speedup against LLVM or GCC was not measured and is not stated.

D. OBSERVED RESULTS

Table 4 is the dashboard sample. At $k = 2$ every algorithm spills `c`, `d`, `t2`, and `t4`. The per-profile estimates are 5850, 5500, and 5200 for Hot loop, Balanced, and Cold-heavy, so the sum is 16550. The static spill operation count of that spill set is 7. Six of the ten live ranges are assigned. At $k = 3$ every algorithm spills only `d`. Each profile contributes 1000, the sum is 3000, and the static count is 1. Nine ranges are assigned. At $k = 4$, $k = 5$, and $k = 6$ every spill set is empty, every per-profile estimate is 0, and the static count is 0. All ten ranges are assigned. The three sums match because the spill sets match, not because the chart copied one algorithm's row. At $k = 6$ the assignments are not identical: static greedy places `t1` on p_0, `t3` on p_1, and `t5` on p_0, while the other two algorithms place `t1` on p_1, `t3` on p_0, and `t5` on p_1. Equation (3) does not see that difference. The assignment footer "10 assigned, 0 spilled, 0 estimated spill operations" is the potential-game row of this response. It is the same fact as the three zeros in the comparison chart.

TABLE 4. Dashboard sample at $\lambda = 3/5$. All three algorithms share each row's spill set, sum, and static count.

k	Sum	Static	Spilled ranges
2	16550	7	`c`, `d`, `t2`, `t4`
3	3000	1	`d`
4	0	0	none
5	0	0	none
6	0	0	none

TABLE 5. Built-in cases with one shared spill set. Sum and static are estimated spill operations.

Case	Sum	Static	Spilled ranges
sequential reuse	0	0	none
diamond	3	3	`b`
loop	40	4	`s`
zero registers	6	6	`a`, `b`, `c`
insufficient	8	8	`b`, `c`, `d`, `f`

The built-in suite has eight cases. Five of them do not separate the algorithms under the case tradeoff. Table 5 lists those five. Sequential reuse spills nothing with one register. The diamond spills `b`. The loop spills `s`, with estimated sum 40 and static count 4, because the profile multiplies a smaller set of sites. Zero registers spills `a`, `b`, and `c`. The insufficient-register clique spills `b`, `c`, `d`, and `f`. Every row passed the interference check, and the three spill sets match. Those rows are validity and cost checks. They are not evidence that the potential search improves on greedy packing.

Three cases do separate a decision. They are plotted in Fig. 6 and listed in Table 6. For the profile-versus-static case, at $\lambda = 0$ and $k = 1$, static greedy spills `b` and the estimated sum is 101. Profile-weighted greedy and the potential search spill `a` and the sum is 8. The static site count prefers `a`; the supplied hot-block counts prefer `b`. For `game_average`, at $\lambda = 0$, all three spill `b` and the sum is 19. The potential search keeps `a`. For `game_worst`, at $\lambda = 1$, static and profile-weighted greedy still spill `b` with sum 19. The potential search spills `a` and the sum is 20. The worst profile falls even though the sum across profiles rises. That is what (5) specifies at $\lambda = 1$. It is not a general win for the search: the summed estimate is higher than the greedy estimate. A unit test locks the kept registers: at $\lambda = 0$ the search assigns p_0 to `a`, and at $\lambda = 1$ it assigns p_0 to `b`.

The illustrative dashboard fixture is a different object. In sample mode the displayed spill operations stay at 4200, 1800, and 1200 regardless of the program text. Those numbers are labeled as a fixture in the interface and in the tests. They are not in Table 4 and they are not measurements.

The zero-cost chart annotation and the empty benchmark request are implementation corrections. The annotation ex-

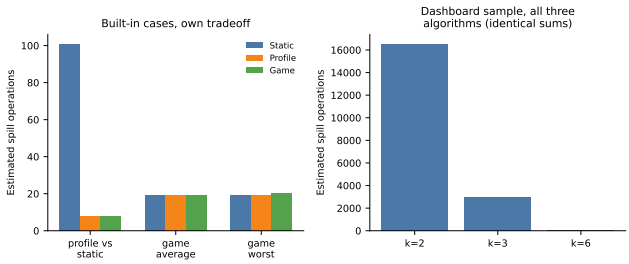


FIGURE 6. Left: estimated spill operations on the three cases that separate a decision, each at its own tradeoff. Right: the dashboard sample, where all three algorithms share the plotted sum.

TABLE 6. Cases in which the spill decision is not identical. Sums are estimated spill operations.

Case	λ	Sum	Spill set by algorithm
profile vs. static	0	101	static: `b`
		8	profile and game: `a`
game average	0	19	all three: `b`
game worst	1	19	static and profile: `b`
		20	game: `a`

plains a returned zero. The empty request stops the dashboard from replacing each case tradeoff with the robustness slider. Neither change creates a new allocation result. Before that request correction, a slider value of 0.6 would have run both `game_average` and `game_worst` at $\lambda = 3/5$, and the potential search spilled `a` in both. The tables in this section use each case's own tradeoff.

E. WHAT IS NOT COMPARED

No paper in Section II reports estimated spill operations on this intermediate representation, with these profiles, under (5). Burroughs reports dynamic store and load counts against LLVM [11]. Pereira and Palsberg report compile time and code quality on SPEC [10]. The RL4ReAl preprint reports LLVM comparisons on x86 and AArch64 [12]. Those setups are not the same experiment. This paper therefore makes no quantitative comparison and no significance claim. A qualitative difference is stated instead. Those systems emit or evaluate machine code, and several of them split or coalesce ranges. PGARA returns a validated assignment and an estimate derived from supplied counts.

The missing experiments are listed in the checklist shipped with this draft. The important ones are repeated here so the results section does not read as a complete evaluation. There is no application suite, no comparison against LLVM or GCC, no repeated timing protocol, no sweep of w_L, w_S, and budgets, and no measurement of code size after spill insertion, because spill code is not inserted. There is no evidence in this repository that the potential search reduces estimated cost on a broad sample of programs. On the programs that were run, it matches greedy except in the $\lambda = 1$ tradeoff case, where its summed estimate is 20 rather than 19.

V. CONCLUSION AND FUTURE WORK

Register allocation is still the problem of keeping simultaneously live values in a finite file and accounting for the rest. Existing coloring, linear-scan, puzzle, profile-distance, evolutionary, and learned allocators address that problem inside much larger compilers, with metrics that this repository does not compute. PGARA implements a smaller contract. It checks a three-address program and explicit profiles, builds may-liveness and an interference graph, and estimates spill operations from load and store sites. Static greedy orders ranges by site counts. Profile-weighted greedy orders them by the sum of profile estimates. The potential search moves one range at a time when the average-to-worst tradeoff strictly falls. The FastAPI service and the React dashboard report those assignments without replacing a failed call by the illustrative fixture.

The verified results are estimates on synthetic programs. On the dashboard sample, all three algorithms spill four ranges at two registers (sum 16550), one range at three registers (sum 3000), and no ranges at four, five, or six registers. On the built-in suite, profile weighting reduces the estimate from 101 to 8 in the hot-block case by spilling a different range, and the potential search at $\lambda = 1$ chooses the range that improves the worst profile while increasing the summed estimate from 19 to 20. Every tabulated assignment passed the interference check. None of these statements is a measured speedup.

The limits are part of the result. The search is local. Identical spill sets hide different register assignments. May-liveness is not path-sensitive. Ranges are not split, copies are not coalesced, and the intermediate representation is not rewritten into spill code. Profiles are trusted as given. Allocator time is recorded and was not evaluated as a performance metric.

Realistic next work stays inside those gaps. A repeated timing protocol on a stated machine would make `execution_time_ms` meaningful. A spill-code rewrite would allow a comparison of emitted loads and stores, which is the quantity Burroughs measured and PGARA does not. Live-range splitting would use the identifier field the data structures already keep. A study on programs other than the eight synthetic cases would show whether the $\lambda = 1$ choice is common or an example. Until those measurements exist,

the supported claim is narrow: given this cost model and these inputs, the three procedures return the assignments and estimates reported above.

AUTHOR CONTRIBUTIONS

Not supplied. The author list, the contribution of each person, and whether a contributor meets the journal's authorship criteria must be added before submission. Contributors who do not meet that criteria belong in the acknowledgment, not in the byline.

ACKNOWLEDGMENT

Personal acknowledgments were not supplied. IEEE Access requires disclosure of generative artificial-intelligence assistance. This draft was prepared from the project repository with the assistance of a coding agent. The human authors must review every citation, number, and sentence and must replace this paragraph with the disclosure they approve before submission.

FUNDING

Funding information was not supplied. No grant number, sponsor, or supporting institution is claimed.

COMPETING INTERESTS

Competing interests were not supplied. No conflict is asserted or denied on behalf of an unnamed author.

DATA AND CODE AVAILABILITY

The analyzer, tests, dashboard, and the result file `verified-results.json` used in Section IV are part of the project that contains this manuscript. The estimates can be recomputed from analyzer version 0.1.0 by the procedure in Section IV-A. A public archive, a DOI for the code, and a data-availability statement approved by the authors were not supplied. No external benchmark suite was downloaded for this draft.

REFERENCES

[1] B. Liu, Y. Huang, J. Gao, J. Shi, Y. Liu, Y. Sun, and W. Ji, "From profiling to optimization: Unveiling the profile guided optimization," arXiv:2507.16649, 2025.
[2] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in International Symposium on Code Generation and Optimization, 2004.
[3] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein, "Register allocation via coloring," Computer Languages, 1981.
[4] G. J. Chaitin, "Register allocation and spilling via graph coloring," in Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction, 1982.
[5] P. Briggs, K. D. Cooper, and L. Torczon, "Improvements to graph coloring register allocation," ACM Transactions on Programming Languages and Systems, 1994.
[6] L. George and A. W. Appel, "Iterated register coalescing," ACM Transactions on Programming Languages and Systems, 1996.
[7] M. Poletto and V. Sarkar, "Linear scan register allocation," ACM Transactions on Programming Languages and Systems, 1999.
[8] O. Traub, G. Holloway, and M. D. Smith, "Quality and speed in linear-scan register allocation," in Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation, 1998.
[9] C. Wimmer and H. Mössenböck, "Optimized interval splitting in a linear scan register allocator," in Proceedings of the 1st ACM/USENIX International Conference on Virtual Execution Environments, 2005.
[10] F. M. Q. Pereira and J. Palsberg, "Register allocation by puzzle solving," in Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation, 2008.
[11] N. Burroughs, "Register allocation and spilling using the expected distance heuristic," Software: Practice and Experience, 2016.
[12] S. VenkataKeerthy, S. Jain, A. Kundu, R. Aggarwal, A. Cohen, and R. Upadrasta, "RL4ReAl: Reinforcement learning for register allocation," arXiv:2204.02013, 2022.
[13] P. Z. da Silva, H. C. d. M. Senefonte, and W. Attrot, "Interference graph dataset for machine learning-based register allocation," IEEE Access, 2024.
[14] G. S. Terci, E. Abdulhalik, A. A. Bayrakci, and B. Boz, "BitEA: BitVertex evolutionary algorithm to enhance performance for register allocation," IEEE Access, 2024.
[15] Y. Oh, I. Jeong, W. W. Ro, and M. K. Yoon, "CASH-RF: A compiler-assisted hierarchical register file in GPUs," IEEE Embedded Systems Letters, 2022.
[16] S. Xiao, J. He, X. Yang, H. Zhou, and Y. Yuan, "Timing error aware register allocation in TS," Computer Systems Science and Engineering, 2022.
[17] S. Xiao, Y. Chen, J. He, and X. Yang, "Read-write dependency aware register allocation," Computer Systems Science and Engineering, 2023.
[18] W. Chen, D. Liu, and S. Liu, "Register allocation compilation technique for ASIP in 5G micro base stations," China Communications, 2022.
[19] Y. Hu, X. Zhang, S. Wang, W. Liang, and K.-C. Li, "Research on global register allocation for code containing array-unit dual-usage register names," Concurrency and Computation: Practice and Experience, 2022.
[20] IEEE Access, "Preparing your article," https://ieeeaccess.ieee.org/authors/preparing-your-article/, 2026, accessed 13 Oct. 2026.
[21] D. Monderer and L. S. Shapley, "Potential games," Games and Economic Behavior, 1996.
[22] Python Software Foundation, "Python 3 documentation," https://docs.python.org/3/, 2026, accessed 13 Oct. 2026.
[23] Pydantic, "Pydantic documentation," https://docs.pydantic.dev/, 2026, accessed 13 Oct. 2026.
[24] FastAPI, "FastAPI documentation," https://fastapi.tiangolo.com/, 2026, accessed 13 Oct. 2026.
[25] React, "React documentation," https://react.dev/, 2026, accessed 13 Oct. 2026.
[26] Vite, "Vite documentation," https://vite.dev/, 2026, accessed 13 Oct. 2026.

• • •

AUTHOR NAME NOT SUPPLIED A biography was not supplied. Replace this paragraph with the corresponding author's biography. Do not add a photograph until one is provided.